Go from Data to Strategy: Tepper School of Business

Probabilistic Model Selection with AIC, BIC, and MDL

by **Jason Brownlee** on August 28, 2020 in **Probability** 64

Share Share Post

Model selection is the problem of choosing one from among a set of candidate models.

It is common to choose a model that performs the best on a hold-out test dataset or to estimate model performance using a resampling technique, such as k-fold cross-validation.

An alternative approach to model selection involves using probabilistic statistical measures that attempt to quantify both the model performance on the training dataset and the complexity of the model. Examples include the Akaike and Bayesian Information Criterion and the Minimum Description Length.

The benefit of these information criterion statistics is that they do not require a hold-out test set, although a limitation is that they do not take the uncertainty of the models into account and may end-up selecting models that are too simple.

In this post, you will discover probabilistic statistics for machine learning model selection.

After reading this post, you will know:

- Model selection is the challenge of choosing one among a set of candidate models.
- Akaike and Bayesian Information Criterion are two ways of scoring a model based on its log-likelihood and complexity.
- Minimum Description Length provides another scoring method from information theory that can be shown to be equivalent to BIC.

Kick-start your project with my new book Probability for Machine Learning, including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

Probabilistic Model Selection Measures AIC, BIC, and MDL
Photo by Guilhem Vellut, some rights reserved.

Overview

This tutorial is divided into five parts; they are:

1. The Challenge of Model Selection
2. Probabilistic Model Selection
3. Akaike Information Criterion
4. Bayesian Information Criterion
5. Minimum Description Length

The Challenge of Model Selection

Model selection is the process of fitting multiple models on a given dataset and choosing one over all others.

> *Model selection: estimating the performance of different models in order to choose the best one.*

— Page 222, The Elements of Statistical Learning, 2016.

This may apply in unsupervised learning, e.g. choosing a clustering model, or supervised learning, e.g. choosing a predictive model for a regression or classification task. It may also be a sub-task of modeling, such as feature selection for a given model.

There are many common approaches that may be used for model selection. For example, in the case of supervised learning, the three most common approaches are:

- Train, Validation, and Test datasets.
- Resampling Methods.
- Probabilistic Statistics.

The simplest reliable method of model selection involves fitting candidate models on a training set, tuning them on the validation dataset, and selecting a model that performs the best on the test dataset according to a chosen metric, such as accuracy or error. A problem with this approach is that it requires a lot of data.

Resampling techniques attempt to achieve the same as the train/val/test approach to model selection, although using a small dataset. An example is k-fold cross-validation where a training set is split into many train/test pairs and a model is fit and evaluated on each. This is repeated for each model and a model is selected with the best average score across the k-folds. A problem with this and the prior approach is that only model performance is assessed, regardless of model complexity.

A third approach to model selection attempts to combine the complexity of the model with the performance of the model into a score, then select the model that minimizes or maximizes the score.

We can refer to this approach as statistical or probabilistic model selection as the scoring method uses a probabilistic framework.

Want to Learn Probability for Machine Learning

Take my free 7-day email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Probabilistic Model Selection

Probabilistic model selection (or “information criteria”) provides an analytical technique for scoring and choosing among candidate models.

Models are scored both on their performance on the training dataset and based on the complexity of the model.

- **Model Performance**. How well a candidate model has performed on the training dataset.
- **Model Complexity**. How complicated the trained candidate model is after training.

Model performance may be evaluated using a probabilistic framework, such as log-likelihood under the framework of maximum likelihood estimation. Model complexity may be evaluated as the number of degrees of freedom or parameters in the model.

> *Historically various 'information criteria' have been proposed that attempt to correct for the bias of maximum likelihood by the addition of a penalty term to compensate for the over-fitting of more complex models.*

— Page 33, Pattern Recognition and Machine Learning, 2006.

A benefit of probabilistic model selection methods is that a test dataset is not required, meaning that all of the data can be used to fit the model, and the final model that will be used for prediction in the domain can be scored directly.

A limitation of probabilistic model selection methods is that the same general statistic cannot be calculated across a range of different types of models. Instead, the metric must be carefully derived for each model.

> *It should be noted that the AIC statistic is designed for preplanned comparisons between models (as opposed to comparisons of many models during automated searches).*

— Page 493, Applied Predictive Modeling, 2013.

A further limitation of these selection methods is that they do not take the uncertainty of the model into account.

> *Such criteria do not take account of the uncertainty in the model parameters, however, and in practice they tend to favour overly simple models.*

— Page 33, Pattern Recognition and Machine Learning, 2006.

There are three statistical approaches to estimating how well a given model fits a dataset and how complex the model is. And each can be shown to be equivalent or proportional to each other, although each was derived from a different framing or field of study.

They are:

- Akaike Information Criterion (AIC). Derived from frequentist probability.
- Bayesian Information Criterion (BIC). Derived from Bayesian probability.
- Minimum Description Length (MDL). Derived from information theory.

Each statistic can be calculated using the log-likelihood for a model and the data. Log-likelihood comes from Maximum Likelihood Estimation, a technique for finding or optimizing the parameters of a model in response to a training dataset.

In Maximum Likelihood Estimation, we wish to maximize the conditional probability of observing the data (*X*) given a specific probability distribution and its parameters (*theta*), stated formally as:

- P(X ; theta)

Where *X* is, in fact, the joint probability distribution of all observations from the problem domain from 1 to n.

- P(x1, x2, x3, …, xn ; theta)

The joint probability distribution can be restated as the multiplication of the conditional probability for observing each example given the distribution parameters. Multiplying many small probabilities together can be unstable; as such, it is common to restate this problem as the sum of the natural log conditional probability.

- sum i to n log(P(xi ; theta))

Given the frequent use of log in the likelihood function, it is commonly referred to as a log-likelihood function.

The log-likelihood function for common predictive modeling problems include the mean squared error for regression (e.g. linear regression) and log loss (binary cross-entropy) for binary classification (e.g. logistic regression).

We will take a closer look at each of the three statistics, AIC, BIC, and MDL, in the following sections.

Akaike Information Criterion

The Akaike Information Criterion, or AIC for short, is a method for scoring and selecting a model.

It is named for the developer of the method, Hirotugu Akaike, and may be shown to have a basis in information theory and frequentist-based inference.

> *This is derived from a frequentist framework, and cannot be interpreted as an approximation to the marginal likelihood.*

— Page 162, Machine Learning: A Probabilistic Perspective, 2012.

The AIC statistic is defined for logistic regression as follows (taken from "The Elements of Statistical Learning"):

- AIC = -2/N * LL + 2 * k/N

Where *N* is the number of examples in the training dataset, *LL* is the log-likelihood of the model on the training dataset, and *k* is the number of parameters in the model.

The score, as defined above, is minimized, e.g. the model with the lowest AIC is selected.

> *To use AIC for model selection, we simply choose the model giving smallest AIC over the set of models considered.*

— Page 231, The Elements of Statistical Learning, 2016.

Compared to the BIC method (below), the AIC statistic penalizes complex models less, meaning that it may put more emphasis on model performance on the training dataset, and, in turn, select more complex models.

> *We see that the penalty for AIC is less than for BIC. This causes AIC to pick more complex models.*

— Page 162, Machine Learning: A Probabilistic Perspective, 2012.

Bayesian Information Criterion

The Bayesian Information Criterion, or BIC for short, is a method for scoring and selecting a model.

It is named for the field of study from which it was derived: Bayesian probability and inference. Like AIC, it is appropriate for models fit under the maximum likelihood estimation framework.

The BIC statistic is calculated for logistic regression as follows (taken from "The Elements of Statistical Learning"):

- BIC = -2 * LL + log(N) * k

Where *log()* has the base-e called the natural logarithm, *LL* is the log-likelihood of the model, *N* is the number of examples in the training dataset, and *k* is the number of parameters in the model.

The score as defined above is minimized, e.g. the model with the lowest BIC is selected.

The quantity calculated is different from AIC, although can be shown to be proportional to the AIC. Unlike the AIC, the BIC penalizes the model more for its complexity, meaning that more complex models will have a worse (larger) score and will, in turn, be less likely to be selected.

> *Note that, compared to AIC […], this penalizes model complexity more heavily.*

— Page 217, Pattern Recognition and Machine Learning, 2006.

Importantly, the derivation of BIC under the Bayesian probability framework means that if a selection of candidate models includes a true model for the dataset, then the probability that BIC will select the true model increases with the size of the training dataset. This cannot be said for the AIC score.

> *… given a family of models, including the true model, the probability that BIC will select the correct model approaches one as the sample size N -> infinity.*

— Page 235, The Elements of Statistical Learning, 2016.

A downside of BIC is that for smaller, less representative training datasets, it is more likely to choose models that are too simple.

Minimum Description Length

The Minimum Description Length, or MDL for short, is a method for scoring and selecting a model.

It is named for the field of study from which it was derived, namely information theory.

Information theory is concerned with the representation and transmission of information on a noisy channel, and as such, measures quantities like entropy, which is the average number of bits required to represent an event from a random variable or probability distribution.

From an information theory perspective, we may want to transmit both the predictions (or more precisely, their probability distributions) and the model used to generate them. Both the predicted target variable and the model can be described in terms of the number of bits required to transmit them on a noisy channel.

The Minimum Description Length is the minimum number of bits, or the minimum of the sum of the number of bits required to represent the data and the model.

> *The Minimum Description Length (MDL) principle recommends choosing the hypothesis that minimizes the sum of these two description lengths.*

— Page 173, Machine Learning, 1997.

The MDL statistic is calculated as follows (taken from "Machine Learning"):

- MDL = L(h) + L(D | h)

Where *h* is the model, *D* is the predictions made by the model, *L(h)* is the number of bits required to represent the model, and *L(D | h)* is the number of bits required to represent the predictions from the model on the training dataset.

The score as defined above is minimized, e.g. the model with the lowest MDL is selected.

The number of bits required to encode (*D* | *h*) and the number of bits required to encode (*h*) can be calculated as the negative log-likelihood; for example (taken from "The Elements of Statistical Learning"):

- MDL = -log(P(theta)) – log(P(y | X, theta))

Or the negative log-likelihood of the model parameters (*theta*) and the negative log-likelihood of the target values (*y*) given the input values (*X*) and the model parameters (*theta*).

This desire to minimize the encoding of the model and its predictions is related to the notion of Occam's Razor that seeks the simplest (least complex) explanation: in this context, the least complex model that predicts the target variable.

> *The MDL principle takes the stance that the best theory for a body of data is one that minimizes the size of the theory plus the amount of information necessary to specify the exceptions relative to the theory …*

— Page 198, Data Mining: Practical Machine Learning Tools and Techniques, 4th edition, 2016.

The MDL calculation is very similar to BIC and can be shown to be equivalent in some situations.

> *Hence the BIC criterion, derived as approximation to log-posterior probability, can also be viewed as a device for (approximate) model choice by minimum description length.*

— Page 236, The Elements of Statistical Learning, 2016.

Worked Example for Linear Regression

We can make the calculation of AIC and BIC concrete with a worked example.

In this section, we will use a test problem and fit a linear regression model, then evaluate the model using the AIC and BIC metrics.

Importantly, the specific functional form of AIC and BIC for a linear regression model has previously been derived, making the example relatively straightforward. In adapting these examples for your own algorithms, it is important to either find an appropriate derivation of the calculation for your model and prediction problem or look into deriving the calculation yourself.

In this example, we will use a test regression problem provided by the make_regression() scikit-learn function. The problem will have two input variables and require the prediction of a target numerical value.

```
...
```

```
# generate dataset
X, y = make_regression(n_samples=100, n_features=2, noise=0.1)
# define and fit the model on all data
```

We will fit a LinearRegression() model on the entire dataset directly.

```
...
# define and fit the model on all data
model = LinearRegression()
model.fit(X, y)
```

Once fit, we can report the number of parameters in the model, which, given the definition of the problem, we would expect to be three (two coefficients and one intercept).

```
...
# number of parameters
num_params = len(model.coef_) + 1
print('Number of parameters: %d' % (num_params))
```

The likelihood function for a linear regression model can be shown to be identical to the least squares function; therefore, we can estimate the maximum likelihood of the model via the mean squared error metric.

First, the model can be used to estimate an outcome for each example in the training dataset, then the mean_squared_error() scikit-learn function can be used to calculate the mean squared error for the model.

```
...
# predict the training set
yhat = model.predict(X)
# calculate the error
mse = mean_squared_error(y, yhat)
print('MSE: %.3f' % mse)
```

Tying this all together, the complete example of defining the dataset, fitting the model, and reporting the number of parameters and maximum likelihood estimate of the model is listed below.

```
# generate a test dataset and fit a linear regression model
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# generate dataset
X, y = make_regression(n_samples=100, n_features=2, noise=0.1)
# define and fit the model on all data
model = LinearRegression()
model.fit(X, y)
# number of parameters
num_params = len(model.coef_) + 1
print('Number of parameters: %d' % (num_params))
# predict the training set
yhat = model.predict(X)
# calculate the error
mse = mean_squared_error(y, yhat)
print('MSE: %.3f' % mse)
```

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example first reports the number of parameters in the model as 3, as we expected, then reports the MSE as about 0.01.

```
Number of parameters: 3
MSE: 0.010
```

Next, we can adapt the example to calculate the AIC for the model.

Skipping the derivation, the AIC calculation for an ordinary least squares linear regression model can be calculated as follows (taken from "A New Look At The Statistical Identification Model", 1974.):

- AIC = n * LL + 2 * k

Where *n* is the number of examples in the training dataset, *LL* is the log-likelihood for the model using the natural logarithm (e.g. the log of the MSE), and *k* is the number of parameters in the model.

The *calculate_aic()* function below implements this, taking *n*, the raw mean squared error (*mse*), and *k* as arguments.

```
# calculate aic for regression
def calculate_aic(n, mse, num_params):
    aic = n * log(mse) + 2 * num_params
    return aic
```

The example can then be updated to make use of this new function and calculate the AIC for the model.

The complete example is listed below.

```python
# calculate akaike information criterion for a linear regression model
from math import log
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# calculate aic for regression
def calculate_aic(n, mse, num_params):
    aic = n * log(mse) + 2 * num_params
    return aic

# generate dataset
X, y = make_regression(n_samples=100, n_features=2, noise=0.1)
# define and fit the model on all data
model = LinearRegression()
model.fit(X, y)
# number of parameters
num_params = len(model.coef_) + 1
print('Number of parameters: %d' % (num_params))
# predict the training set
yhat = model.predict(X)
# calculate the error
mse = mean_squared_error(y, yhat)
print('MSE: %.3f' % mse)
# calculate the aic
aic = calculate_aic(len(y), mse, num_params)
print('AIC: %.3f' % aic)
```

Running the example reports the number of parameters and MSE as before and then reports the AIC.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

In this case, the AIC is reported to be a value of about -451.616. This value can be minimized in order to choose better models.

```
Number of parameters: 3
MSE: 0.010
AIC: -451.616
```

We can also explore the same example with the calculation of BIC instead of AIC.

Skipping the derivation, the BIC calculation for an ordinary least squares linear regression model can be calculated as follows (taken from here):

- BIC = n * LL + k * log(n)

Where n is the number of examples in the training dataset, *LL* is the log-likelihood for the model using the natural logarithm (e.g. log of the mean squared error), and *k* is the number of parameters in the model, and *log()* is the natural logarithm.

The *calculate_bic()* function below implements this, taking *n*, the raw mean squared error (*mse*), and *k* as arguments.

```python
# calculate bic for regression
def calculate_bic(n, mse, num_params):
    bic = n * log(mse) + num_params * log(n)
    return bic
```

The example can then be updated to make use of this new function and calculate the BIC for the model.

The complete example is listed below.

```python
# calculate bayesian information criterion for a linear regression model
from math import log
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# calculate bic for regression
def calculate_bic(n, mse, num_params):
    bic = n * log(mse) + num_params * log(n)
    return bic

# generate dataset
X, y = make_regression(n_samples=100, n_features=2, noise=0.1)
# define and fit the model on all data
model = LinearRegression()
model.fit(X, y)
# number of parameters
num_params = len(model.coef_) + 1
print('Number of parameters: %d' % (num_params))
# predict the training set
yhat = model.predict(X)
# calculate the error
mse = mean_squared_error(y, yhat)
print('MSE: %.3f' % mse)
# calculate the bic
bic = calculate_bic(len(y), mse, num_params)
print('BIC: %.3f' % bic)
```

Running the example reports the number of parameters and MSE as before and then reports the BIC.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

In this case, the BIC is reported to be a value of about -450.020, which is very close to the AIC value of -451.616. Again, this value can be minimized in order to choose better models.

```
Number of parameters: 3
MSE: 0.010
BIC: -450.020
```

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Books

- Chapter 7 Model Assessment and Selection, The Elements of Statistical Learning, 2016.
- Section 1.3 Model Selection, Pattern Recognition and Machine Learning, 2006.
- Section 4.4.1 Model comparison and BIC, Pattern Recognition and Machine Learning, 2006.
- Section 6.6 Minimum Description Length Principle, Machine Learning, 1997.
- Section 5.3.2.4 BIC approximation to log marginal likelihood, Machine Learning: A Probabilistic Perspective, 2012.
- Applied Predictive Modeling, 2013.
- Section 28.3 Minimum description length (MDL), Information Theory, Inference and Learning Algorithms, 2003.
- Section 5.10 The MDL Principle, Data Mining: Practical Machine Learning Tools and Techniques, 4th edition, 2016.

Papers

- A New Look At The Statistical Identification Model, 1974.

API

- sklearn.datasets.make_regression API.
- sklearn.linear_model.LinearRegression API.
- sklearn.metrics.mean_squared_error API.

Articles

- Akaike information criterion, Wikipedia.
- Bayesian information criterion, Wikipedia.
- Minimum description length, Wikipedia.

Summary

In this post, you discovered probabilistic statistics for machine learning model selection.

Specifically, you learned:

- Model selection is the challenge of choosing one among a set of candidate models.
- Akaike and Bayesian Information Criterion are two ways of scoring a model based on its log-likelihood and complexity.
- Minimum Description Length provides another scoring method from information theory that can be shown to be equivalent to BIC.

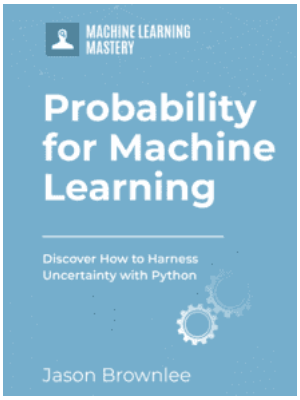
Do you have any questions?
Ask your questions in the comments below and I will do my best to answer.

Get a Handle on Probability for Machine Learning!

Develop Your Understanding of Probability

...with just a few lines of python code

Discover how in my new Ebook:
Probability for Machine Learning



It provides **self-study tutorials** and **end-to-end projects** on:
Bayes Theorem, *Bayesian Optimization*, *Distributions*, *Maximum Likelihood*, *Cross-Entropy*, *Calibrating Models*
and much more...

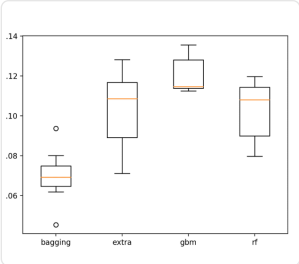
Finally Harness Uncertainty in Your Projects

Skip the Academics. Just Results.

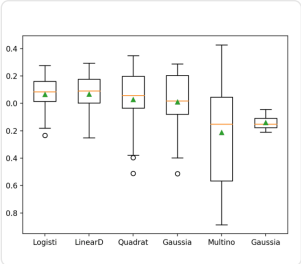
SEE WHAT'S INSIDE

Share Share Post

More On This Topic



Probabilistic Forecasting Model to Predict Air…



How to Develop a Probabilistic Model of Breast…

The Model Selection Showdown: 6 Considerations for…

Model Selection Tips From Competitive Machine Learning

A Gentle Introduction to Model Selection for Machine…

Feature Selection to Improve Accuracy and Decrease…

About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.
View all posts by Jason Brownlee →

‹ A Gentle Introduction to Logistic Regression With Maximum Likelihood Estimation

A Gentle Introduction to Expectation-Maximization (EM Algorithm) ›